OS LAB PROGRAMS

FCFS :

```c
#include <stdio.h>
int main() {
    int n, i;
    float totalTAT = 0, totalWT = 0;
    printf("Enter number of processes: \n");
    scanf("%d", &n);
    int at[n], bt[n], ct[n], tat[n], wt[n];
    for(i = 0; i < n; i++) {
        printf("Enter Arrival Time and Burst Time for P%d: \n", i+1);
        scanf("%d %d", &at[i], &bt[i]);
    }
    ct[0] = at[0] + bt[0];
    for(i = 1; i < n; i++) {
        if(ct[i-1] < at[i])
            ct[i] = at[i] + bt[i];
        else
            ct[i] = ct[i-1] + bt[i];
    }
    for(i = 0; i < n; i++) {
        tat[i] = ct[i] - at[i];
        wt[i] = tat[i] - bt[i];
        totalTAT += tat[i];
        totalWT += wt[i];
    }
    printf("\nAverage Turnaround Time: %.2f\n", totalTAT/n);
    printf("Average Waiting Time: %.2f", totalWT/n);
    return 0;
}
```

FirstFit:

```c
#include <stdio.h>
int main() {
    int m, n, i, j;
    printf("Enter number of memory blocks:\n");
    scanf("%d", &m);
    int block[m], original[m];
    printf("Enter size of each block:\n");
    for(i = 0; i < m; i++) {
        scanf("%d", &block[i]);
        original[i] = block[i];
    }
    printf("Enter number of processes:\n");
    scanf("%d", &n);
    int process[n];
    printf("Enter size of each process:\n");
    for(i = 0; i < n; i++) {
        scanf("%d", &process[i]);
    }
    for(i = 0; i < n; i++) {
        int found = 0;
        for(j = 0; j < m; j++) {
            if(block[j] >= process[i]) {
                int fragment = block[j] - process[i];
                printf("Process %d of size %d is allocated to Block %d of size %d with Fragment %d\n"
                       i+1, process[i], j+1, original[j], fragment);
                block[j] -= process[i];
                found = 1;
                break;
            }
        }
        if(!found) {
            printf("Process %d of size %d is not allocated\n", i+1, process[i]);
        }
    }
    return 0;
}
```

BestFit:

```c
#include <stdio.h>
int main() {
    int m, n, i, j;
    printf("Enter number of memory blocks:\n");
    scanf("%d", &m);
    int block[m], original[m];
    printf("Enter size of each block:\n");
    for(i = 0; i < m; i++) {
        scanf("%d", &block[i]);
        original[i] = block[i];
    }
    printf("Enter number of processes:\n");
    scanf("%d", &n);
    int process[n];
    printf("Enter size of each process:\n");
    for(i = 0; i < n; i++) {
        scanf("%d", &process[i]);
    }
    for(i = 0; i < n; i++) {
        int bestIdx = -1;
        for(j = 0; j < m; j++) {
            if(block[j] >= process[i]) {
                if(bestIdx == -1 || block[j] < block[bestIdx]) {
                    bestIdx = j;
                }
            }
        }
        if(bestIdx != -1) {
            int fragment = block[bestIdx] - process[i];
            printf("Process %d of size %d is allocated to Block %d of size %d with Fragment %d\n",
                   i+1, process[i], bestIdx+1, original[bestIdx], fragment);
            block[bestIdx] -= process[i];
        } else {
            printf("Process %d of size %d is not allocated\n", i+1, process[i]);
        }
    }
    return 0;
}
```

FIFO:

```c
#include <stdio.h>
int main() {
    int n, f, i, j, k;
    printf("Enter number of pages in reference string: \n");
    scanf("%d", &n);
    int pages[n];
    printf("Enter the reference string: \n");
    for(i = 0; i < n; i++) {
        scanf("%d", &pages[i]);
    }
    printf("Enter number of frames: \n");
    scanf("%d", &f);
    int frame[f];
    int front = 0;
    int faults = 0, hits = 0;
    for(i = 0; i < f; i++) {
        frame[i] = -1;
    }
    for(i = 0; i < n; i++) {
        int found = 0;
        for(j = 0; j < f; j++) {
            if(frame[j] == pages[i]) {
                found = 1;
                hits++;
                break;
            }
        }
        if(found == 0) {
            frame[front] = pages[i];
            front = (front + 1) % f;
            faults++;
        }
    }
    printf("\nTotal Page Faults: %d", faults);
    printf("\nTotal Page Hits: %d\n", hits);
    return 0;
}
```

LRU:

```c
#include <stdio.h>
int main() {
    int n, f, i, j;
    printf("Enter number of pages in reference string: \n");
    scanf("%d", &n);
    int pages[n];
    printf("Enter the reference string: \n");
    for(i = 0; i < n; i++) {
        scanf("%d", &pages[i]);
    }
    printf("Enter number of frames: \n");
    scanf("%d", &f);
    int frame[f], time[f];
    int faults = 0, hits = 0, counter = 0;
    for(i = 0; i < f; i++) {
        frame[i] = -1;
        time[i] = 0;
    }
    for(i = 0; i < n; i++) {
        int found = -1;
        for(j = 0; j < f; j++) {
            if(frame[j] == pages[i]) {
                found = j;
                hits++;
                counter++;
                time[j] = counter;
                break;
            }
        }
        if(found == -1) {
            int lru = 0;
            for(j = 1; j < f; j++) {
                if(time[j] < time[lru]) {
                    lru = j;
                }
            }
            frame[lru] = pages[i];
            counter++;
            time[lru] = counter;
            faults++;
        }
    }
    printf("\nTotal Page Faults: %d", faults);
    printf("\nTotal Page Hits: %d\n", hits);
    return 0;
}
```

RR:

```c
#include <stdio.h>
int main() {
    int n, tq, i;
    printf("Enter number of processes: \n");
    scanf("%d", &n);
    printf("Enter Time Quantum: \n");
    scanf("%d", &tq);
    int at[n], bt[n], rt[n];
    int ct[n], tat[n], wt[n];
    int completed = 0, time = 0;
    for(i = 0; i < n; i++) {
        printf("Enter AT and BT for P%d: \n", i+1);
        scanf("%d %d", &at[i], &bt[i]);
        rt[i] = bt[i];
    }
    while(completed < n) {
        int executed = 0;
        for(i = 0; i < n; i++) {
            if(at[i] <= time && rt[i] > 0) {
                executed = 1;
                if(rt[i] > tq) {
                    time += tq;
                    rt[i] -= tq;
                } else {
                    time += rt[i];
                    rt[i] = 0;
                    ct[i] = time;
                    completed++;
                }
            }
        }
        if(executed == 0) {
            time++;
        }
    }
    float total_tat = 0, total_wt = 0;
    for(i = 0; i < n; i++) {
        tat[i] = ct[i] - at[i];
        wt[i] = tat[i] - bt[i];
        total_tat += tat[i];
        total_wt += wt[i];
    }
    printf("\nAverage Turnaround Time: %.2f", total_tat / n);
    printf("\nAverage Waiting Time: %.2f\n", total_wt / n);
    return 0;
}
```

SJF:

```c
#include <stdio.h>
int main() {
    int n,i,j,time=0,min,p;
    float avgTAT=0,avgWT=0;
    printf("Enter number of processes: \n");
    scanf("%d",&n);
    int at[n],bt[n],ct[n],tat[n],wt[n],done[n];
    for(i=0;i<n;i++){
        printf("Enter Arrival Time and Burst Time for P%d: \n",i+1);
        scanf("%d %d",&at[i],&bt[i]);
        done[i]=0;
    }
    for(i=0;i<n;i++){
        min=999;
        for(j=0;j<n;j++){
            if(at[j]<=time && done[j]==0 && bt[j]<min){
                min=bt[j];
                p=j;
            }
        }
        if(min==999){
            time++;
            i--;
        }else{
            time=time+bt[p];
            ct[p]=time;
            done[p]=1;
        }
    }
    for(i=0;i<n;i++){
        tat[i]=ct[i]-at[i];
        wt[i]=tat[i]-bt[i];
        avgTAT+=tat[i];
        avgWT+=wt[i];
    }
    printf("\nAverage Turnaround Time = %.2f\n",avgTAT/n);
    printf("Average Waiting Time = %.2f",avgWT/n);
    return 0;
}
```

BANKER'S ALGORITHM:

```c
#include <stdio.h>
int main()
{
    int n, r, i, j, k;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    printf("Enter number of resource types: ");
    scanf("%d", &r);
    int alloc[n][r], max[n][r], need[n][r], avail[r];
    int finish[n], safe_sequence[n];
    int count = 0, safeFound;
    printf("Enter Allocation Matrix:\n");
    for(i = 0; i < n; i++)
    {
        for(j = 0; j < r; j++)
        {
            scanf("%d", &alloc[i][j]);
        }
    }
    printf("Enter Max Matrix:\n");
    for(i = 0; i < n; i++)
    {
        for(j = 0; j < r; j++)
        {
            scanf("%d", &max[i][j]);
        }
    }
```

```c
printf("Enter Available Resources:\n");
for(i = 0; i < r; i++)
{
    scanf("%d", &avail[i]);
}
for(i = 0; i < n; i++)
{
    finish[i] = 0;
}
for(i = 0; i < n; i++)
{
    for(j = 0; j < r; j++)
    {
        need[i][j] = max[i][j] - alloc[i][j];
    }
}
while(count < n)
{
    safeFound = 0;
    for(i = 0; i < n; i++)
    {
        if(finish[i] == 0)
        {
            int safe = 1;
            for(j = 0; j < r; j++)
            {
                if(need[i][j] > avail[j])
                {
                    safe = 0;
                    break;
                }
            }
            if(safe)
            {
                for(k = 0; k < r; k++)
                {
                    avail[k] = avail[k] + alloc[i][k];
                }
```

```
                }
                safe_sequence[count] = i;
                finish[i] = 1;
                safeFound = 1;
                count++;
            }
        }
    }
    if(!safeFound)
    {
        printf("System is in unsafe state! May occur deadlock\n");
        return 0;
    }
}
printf("Safe sequence is: ");
for(i = 0; i < n; i++)
{
    printf("P%d ", safe_sequence[i]);
}

return 0;
}
```

PRODUCER CONSUMER PROBLEM:

(USING SEMAPHORES)

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#define BUFFER_SIZE 5
int buffer[BUFFER_SIZE];
int in = 0, out = 0;
sem_t mutex, empty, full;
void* producer(void* arg) {
    int item;
    for (int i = 0; i < 10; i++) {
        item = i;
        sem_wait(&empty);
        sem_wait(&mutex);
        buffer[in] = item;
        printf("Produced: %d\n", item);
        in = (in + 1) % BUFFER_SIZE;
        sem_post(&mutex);
        sem_post(&full);
        sleep(1);
    }
    return NULL;
}
void* consumer(void* arg) {
    int item;
    for (int i = 0; i < 10; i++) {
        sem_wait(&full);
        sem_wait(&mutex);
        item = buffer[out];
        printf("Consumed: %d\n", item);
        out = (out + 1) % BUFFER_SIZE;
        sem_post(&mutex);
        sem_post(&empty);
        sleep(2);
    }
    return NULL;
}
```

```c
int main() {
    pthread_t p, c;
    sem_init(&mutex, 0, 1);
    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);
    pthread_create(&p, NULL, producer, NULL);
    pthread_create(&c, NULL, consumer, NULL);
    pthread_join(p, NULL);
    pthread_join(c, NULL);
    sem_destroy(&mutex);
    sem_destroy(&empty);
    sem_destroy(&full);
    return 0;
}
```

PRIORITY:

```c
#include <stdio.h>
int main() {
    int n, i, j, time = 0, completed = 0;
    float total_wt = 0, total_tat = 0;
    printf("Enter number of processes: \n");
    scanf("%d", &n);
    int at[n], bt[n], pr[n];
    int wt[n], tat[n], ct[n], done[n];
    for(i = 0; i < n; i++) {
        printf("Enter AT, BT and Priority for P%d: \n", i+1);
        scanf("%d %d %d", &at[i], &bt[i], &pr[i]);
        done[i] = 0; // not completed
    }
    while(completed < n) {
        int idx = -1;
        int highest_priority = 9999;
        for(i = 0; i < n; i++) {
            if(at[i] <= time && done[i] == 0) {
                if(pr[i] < highest_priority) {
                    highest_priority = pr[i];
                    idx = i;
                }
                if(pr[i] == highest_priority) {
                    if(at[i] < at[idx]) {
                        idx = i;
                    }
                }
            }
        }
        if(idx != -1) {
            time += bt[idx];
            ct[idx] = time;
            tat[idx] = ct[idx] - at[idx];
            wt[idx] = tat[idx] - bt[idx];
            total_wt += wt[idx];
            total_tat += tat[idx];
            done[idx] = 1;
            completed++;
        } else {
            time++;
        }
    }
    printf("\nAverage Turnaround Time = %.2f", total_tat / n);
    printf("\nAverage Waiting Time = %.2f\n", total_wt / n);

    return 0;
}
```

IPC USING SHARED MEMORY:

(I) Sender.c

```c
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#define SharedMemSize 50
int main()
{
    int shmid;
    key_t key;
    char *shared_memory;
    key = 5677;
    // Create shared memory segment
    if ((shmid = shmget(key, SharedMemSize, IPC_CREAT | 0666)) < 0)
    {
        perror("shmget");
        exit(1);
    }
    // Attach the segment
    if ((shared_memory = shmat(shmid, NULL, 0)) == (char *) -1)
    {
        perror("shmat");
        exit(1);
    }
    // Write message to shared memory
    sprintf(shared_memory, "Welcome to Shared Memory");
    // Give time for receiver to read
    sleep(2);

    return 0;
}
```

(II) Receiver.c

```c
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <stdio.h>
#include <stdlib.h
#define SharedMemSize 5
int main()
{
    int shmid;
    key_t key;
    char *shared_memory;
    key = 5677;
    // Get shared memory segment
    if ((shmid = shmget(key, SharedMemSize, 0666)) < 0)
    {
        perror("shmget");
        exit(1);
    }
    // Attach the segment
    if ((shared_memory = shmat(shmid, NULL, 0)) == (char *) -1)
    {
        perror("shmat");
        exit(1);
    }
    // Read message from shared memory
    printf("Message Received: %s\n", shared_memory);
    return 0;
}
```